Data Structure Choices — Design Analysis

1. `ScheduleServiceImpl`

1.1. `moduleMap` — `LinkedHashMap<String, StudyModule>`

During the BFS loop, each placed module needs to be looked up by its key string. A `Map` does this in $O(1)$. A `Set` would require scanning all elements to find a match — $O(V)$ per lookup — which would degrade the total algorithm from $O(V + E)$ to $O(V^2 + E)$.

`LinkedHashMap` is used instead of plain `HashMap` so modules are processed in the order they were created, making the output deterministic: the same input always produces the same schedule.

`"Math"`	StudyModule(key: `"Math"`, ects: 4, …)
`"OO"`	StudyModule(key: `"OO"`, ects: 4, …)
`"DB1"`	StudyModule(key: `"DB1"`, ects: 4, …)
⋮ (insertion order)	⋮
`"Thesis"`	StudyModule(key: `"Thesis"`, ects: 12, …)

Table 1: moduleMap — every key maps to its StudyModule

1.2. `inDegree` — `LinkedHashMap<String, Integer>`

Stores how many prerequisites each module still has left to satisfy. `LinkedHashMap` is used for the same reason as `moduleMap`: insertion order makes the initial ready-queue seed deterministic.

`"Math"`	`0`	no prerequisites → enters ready queue immediately
`"OO"`	`0`	no prerequisites → enters ready queue immediately
`"DB1"`	`1`	needs OO
`"CPI"`	`2`	needs OO, Math
`"SE1"`	`3`	needs AD1, CPI, DB1
⋮	⋮	
`"Thesis"`	`3`	needs DB2, SE2, UI2

Table 2: inDegree — count of unsatisfied in-program prerequisites; a module enters the ready queue when its count reaches 0

1.3. `dependents` — `HashMap<String, List<String>>`

Stores, for each module, the list of modules that depend on it. This is used to know which in-degrees to decrement when a module is placed. The order of decrements does not affect correctness, so a plain `HashMap` is sufficient.

"Math"	["CPI"]
"OO"	["DB1", "AD1", "AD2", "CPI"]
"DB1"	["DB2", "SE1", "SE2"]
"AD1"	["AD2", "SE1", "UI1"]
⋮	⋮
"Thesis"	[]

Table 3: dependents — reverse adjacency list; when a module is placed, each entry in its list has its inDegree decremented

1.4. `ready` — `ArrayDeque<String>`

Holds module keys that are ready to be scheduled (all prerequisites satisfied). The variable is declared as `Queue<String>` and backed by `ArrayDeque` — only the queue-side operations are used: `add` (enqueue at back), `isEmpty`, `clear`, and iteration via `List.copyOf`.

Why a queue at all? A queue models the BFS layer boundary naturally: new modules are enqueued at the back as their in-degrees reach zero, and the front-to-back snapshot at the start of each iteration captures exactly the current ready layer without mixing in modules that become ready mid-iteration.

Why `ArrayDeque` and not `LinkedList`? Both implement `Queue`, but `LinkedList` allocates a separate `Node` object for every element. `ArrayDeque` stores elements in a circular array, so elements are contiguous in memory.

front → | "Math" | "OO" | ← back

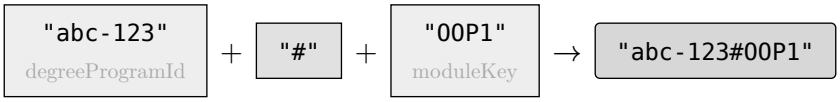
Listing 1: ready — initial state after seeding; contains all modules whose inDegree = 0

2. Repository Layer

Both repositories use a `LinkedHashMap` as their backing store. This gives $O(1)$ for all four operations (save, find, findAll, delete) and preserves insertion order, so `findAll()` returns entries in creation order — producing a stable, predictable list in the UI.

`InMemoryStudyModuleRepository` needs to store modules from multiple degree programs in a single map. The alternative would be a nested map: `Map<String, Map<String, StudyModule>>` — an outer map keyed by degree program ID, each containing an inner map keyed by module key. Instead, a **flat map with a composite key** is used: the two IDs are joined into a single string (`"degreeProgramId#moduleKey"`) and used as the map key directly. For example, a module with key `"OOP1"` in program `"abc-123"` is stored under the key `"abc-123#OOP1"`. A flat map is simpler: there is only one map to create, one map to iterate, and no need to manage the lifecycle of inner maps.

The `#` separator is safe because degree-program IDs are UUIDs and module keys are short alphanumeric strings — neither can contain `#`, so two different (program, module) pairs can never produce the same composite key.



Listing 2: Composite key — degreeProgramId and moduleKey joined by `"#"` form a single flat-map key that uniquely identifies a (program, module) pair

3. Model Layer

3.1. `Schedule` and `Semester` — `unmodifiableList` view

Both classes build their lists incrementally: the BFS loop adds semesters to `Schedule` and modules to each `Semester` one by one. At the same time, their getters are public, so external code could potentially call `getSemesters().add(...)` and corrupt the state mid-calculation.

Wrapping the list in `Collections.unmodifiableList` before returning it blocks that: any attempt to add, remove, or clear via the returned reference throws an exception. The BFS itself keeps a direct reference to the underlying `ArrayList` and continues adding to it freely — only the view handed to callers is read-only. No copying happens, so this protection costs $O(1)$.



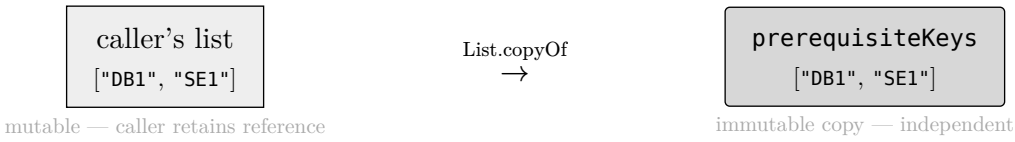
Listing 3: The unmodifiable wrapper exposes a read-only view of the ArrayList; the BFS keeps a direct reference to the underlying list and continues to add to it

3.2. `StudyModule` prerequisite keys — `List.copyOf`

`StudyModule` is constructed by the controller, which builds the prerequisite list from form input and passes it in. If the constructor stored that reference directly, the controller could modify the list afterwards (accidentally or otherwise) and silently change a module it already saved.

`List.copyOf` makes a new, independent, immutable list at construction time, severing any connection to the original. An `unmodifiableList` wrapper would not be enough here: it wraps the caller's list rather than copying it, so the caller would still hold a mutable reference to the underlying data.

The key difference between the two cases: `Schedule`/`Semester` own their lists from the start and just need to hide them from callers, so a wrapper is enough. `StudyModule` receives a list it does not own, so it must copy it.



Listing 4: `List.copyOf` creates an independent immutable copy at construction time, severing any link to the caller's original list